

Tema 5 (Parte 3). Agrupaciones

1. Agrupaciones
2. Funciones de cálculo con grupos
3. Condición de agrupamiento (Having)
4. Operaciones entre consultas



1. Agrupaciones

Recordemos la sintaxis y el orden en el que se ejecutan cada una de las cláusulas del SELECT:

Sintaxis

```
SELECT
  [DISTINCT] select_expr [, select_expr ...]
  [ FROM table_references
    [WHERE where_condition]
    [GROUP BY {col_name | expr | position} [ASC | DESC], ... [WITH ROLLUP]]
    [HAVING where_condition]
    [ORDER BY {col_name | expr | position} [ASC | DESC], ...]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
```

Orden en el que se ejecutan las cláusulas del SELECT:

Nos centramos en



Cláusula	Obligatoria / Opcional	Expresión
FROM	Obligatoria	Tabla(s) fuente(s)
WHERE	Opcional	Condición (condiciones) o restricción (restricciones)
GROUP BY	Opcional	Agrupación (agrupaciones)
HAVING	Opcional	Condición (condiciones) o restricción (restricciones) sobre la(s) agrupación (agrupaciones)
SELECT	Obligatoria	Lista columna(s) y/o elementos de extracción
ORDER BY	Opcional	Ordenación (ordenaciones)
LIMIT	Opcional	Limitar el número de filas devueltas en la consulta

1. Agrupaciones

Es muy común realizar consultas en las que se deseen agrupar los datos a fin de realizar cálculos “en vertical”, es decir, calculados a partir de datos de distintas filas. Para ello, se utiliza la cláusula **GROUP BY**, que **permite indicar en base a qué registros se realiza una agrupación**.

En el apartado **GROUP BY** **se indican las columnas (una o varias) por las que se desea agrupar. La función de este apartado es crear un único registro por cada valor distinto en las columnas del grupo**. Es decir, todas las filas que tienen el mismo valor en la columna de agrupación se convierten en una.

Ejemplo:

Si sobre la tabla *existencias* que se muestra a continuación quisiéramos agrupar en base a las columnas *tipo* y *modelo* (como se especifica en la siguiente sentencia), se creará un único registro por cada tipo y modelo distintos:

```
SELECT tipo, modelo
```

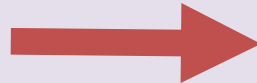
```
FROM existencias
```

```
GROUP BY tipo, modelo;
```

La tabla de *existencias* sin agrupar sería:

TIPO	MODELO	N_ALMACEN	CANTIDAD
AR	6	1	2500
AR	6	2	5600
AR	6	3	2430
AR	9	1	250
AR	9	2	4000
AR	9	3	678
AR	15	1	5667
AR	20	3	43
BI	10	2	340
BI	10	3	23
BI	38	1	1100
BI	38	2	540

Crea esta salida:



TIPO	MODELO
AR	6
AR	9
AR	15
AR	20
BI	10
BI	38

Es decir, **se puede ver como un resumen** de los datos anteriores según las condiciones de agrupación que hemos establecido.

1. Agrupaciones

Siguiendo con el ejemplo anterior, los datos *n_almacen* y *cantidad* no están disponibles directamente, ya que son distintos en los registros del mismo grupo.

Por ***ejemplo***, esta consulta sería **errónea**:

```
SELECT tipo, modelo, cantidad  
FROM existencias  
GROUP BY tipo, modelo;
```

Ya que, ¿qué valor de *cantidad* se cogería para cada fila de *tipo-modelo*? Es decir, para la fila “AR – 6”, habría 3 cantidades posibles: 2500, 5600 y 2430, lo que no puede ser.

Sin embargo, sí que **se podrían utilizar funciones que realicen operaciones con esas columnas**, como **calcular la cantidad media o la cantidad total de cada grupo**.

Ejemplo:

```
SELECT tipo, modelo, SUM(cantidad)  
FROM existencias  
GROUP BY tipo, modelo;
```

En este caso, para la fila “AR – 6”, sí tenemos un único valor de SUM(cantidad), que sería 10530.

2. Funciones de cálculo con grupos

La creación de grupos es muy interesante por las posibilidades de cálculo que ofrece. Para ello se utilizan funciones que permiten trabajar con los registros de un grupo.

FUNCIÓN	DESCRIPCIÓN
COUNT (* columna)	Cuenta los elementos de un grupo. - Se utiliza el asterisco para indicar que cuente el número de valores de una fila (incluyendo los nulos). - Si se pone una columna, sólo contará el número de valores de una columna (excepto los nulos). Ejem: <code>SELECT COUNT(*) FROM emple WHERE comision IS NOT NULL;</code> → 4
SUM (expresión)	Suma los valores de la expresión Ejem: <code>SELECT SUM(SALARIO) FROM emple WHERE comision IS NOT NULL;</code> → 6075
AVG (expresión)	Calcula la media aritmética sobre la expresión indicada Ejem: <code>SELECT AVG(SALARIO) FROM emple WHERE comision IS NOT NULL;</code> → 1518.7500
MIN (expresión)	Mínimo valor que toma la expresión indicada Ejem: <code>SELECT MIN(salario) FROM emple WHERE comision IS NOT NULL;</code> → 1350
MAX (expresión)	Máximo valor que toma la expresión indicada Ejem: <code>SELECT MAX(salario) FROM emple WHERE comision IS NOT NULL;</code> → 1625
STDDEV (expresión)	Calcula la desviación estándar
VARIANCE (expresión)	Calcula la varianza

2. Funciones de cálculo con grupos

Todas las funciones de la tabla anterior se calculan para cada grupo. Así, la expresión:

```
SELECT tipo, modelo, SUM(Cantidad)
FROM existencias
GROUP BY tipo, modelo;
```

Obtiene este resultado:

TIPO	MODELO	SUM(CANTIDAD)
AR	6	10530
AR	9	4928
AR	15	5667
AR	20	43
BI	10	363
BI	38	1740

Ahora bien, **¿cómo se deben utilizar de forma adecuada estas funciones?** Trabajemos sobre un ejemplo. Imaginemos que queremos devolver el listado con los productos que tienen un precio superior al precio medio de todos los artículos que existen en una tabla productos. Para resolver esta consulta, podríamos pensar en realizar una consulta como:

```
SELECT * FROM producto WHERE precio > AVG(precio);
```

Sin embargo, esta sentencia da un **error** por un “uso inválido de la función de agrupación”. Esto se debe a que **las funciones de agregación no se pueden utilizar en la cláusula WHERE**. La forma correcta de realizar lo pedido sería:

```
SELECT * FROM producto WHERE precio > (SELECT AVG(precio) FROM producto);
```

2. Funciones de cálculo con grupos

MODIFICADOR “WITH ROLLUP”

Existe un modificador propio de la sentencia GROUP BY, “WITH ROLLUP”, que **permite añadir filas extra al conjunto de resultados devuelto para representar resúmenes “super-agregados”**.

La columna “super-agregada” **se representa por NULL**.

Si hay múltiples columnas de agregación en la sentencia GROUP BY, entonces se generarán múltiples agregaciones sobre diferentes columnas.

WITH ROLLUP no puede ser utilizada con ORDER BY. Para ordenar, se puede utilizar la ordenación a través de las columnas de la cláusula GROUP BY, aunque, en este caso, las filas “super-agregadas” siempre se añadirán al final.

Ejemplos:

Se desea conocer la suma de salarios por oficios. Para ello, haríamos:

SELECT oficio, SUM(salario) FROM emple GROUP BY oficio;

oficio	SUM(salario)
ANALISTA	6000
DIRECTOR	8790
EMPLEADO	5495
PRESIDENTE	4100
VENDEDOR	6075

Si además **quisiéramos saber la suma total de todos los salarios, podríamos obtenerlo a través de una consulta “super-agregada”**, es decir, se podría utilizar el modificador WITH ROLLUP tal y como se muestra a continuación:

SELECT oficio, SUM(salario) FROM emple GROUP BY oficio WITH ROLLUP;

oficio	SUM(salario)
ANALISTA	6000
DIRECTOR	8790
EMPLEADO	5495
PRESIDENTE	4100
VENDEDOR	6075
NULL	30460

2. Funciones de cálculo con grupos

MODIFICADOR “WITH ROLLUP”

Esta misma idea se puede llevar a **agregaciones por más de un campo**. Por ejemplo, si quisiéramos obtener la suma de salarios por oficio y departamento, haríamos:

SELECT oficio, dept_no, SUM(salario) FROM emple GROUP BY oficio, dept_no;

oficio	dept_no	SUM(salario)
ANALISTA	20	6000
DIRECTOR	10	2885
DIRECTOR	20	2900
DIRECTOR	30	3005
EMPLEADO	10	1690
EMPLEADO	20	2470
EMPLEADO	30	1335
PRESIDENTE	10	4100
VENDEDOR	30	6075

Si, además, quisiéramos conocer esta suma de salarios por oficio, independientemente del departamento, o por el agrupado de oficio y departamento, podríamos ejecutar la misma sentencia que en el caso anterior, pero como el modificador WITH ROLLUP.

SELECT oficio, dept_no, SUM(salario) FROM emple GROUP BY oficio, dept_no WITH ROLLUP;

oficio	dept_no	SUM(salario)
ANALISTA	20	6000
ANALISTA	NULL	6000
DIRECTOR	10	2885
DIRECTOR	20	2900
DIRECTOR	30	3005
DIRECTOR	NULL	8790
EMPLEADO	10	1690
EMPLEADO	20	2470
EMPLEADO	30	1335
EMPLEADO	NULL	5495
PRESIDENTE	10	4100
PRESIDENTE	NULL	4100
VENDEDOR	30	6075
VENDEDOR	NULL	6075
NULL	NULL	30460

Podemos observar que la columna NULL identifica el “super-agregado” de unos valores, por ejemplo, la columna “DIRECTOR – NULL – 8790” identifica la suma de salarios de los directores, independientemente del departamento al que pertenezcan.

2. Funciones de cálculo con grupos

ORDENACIÓN EN LA SALIDA

Otro aspecto destacado de la cláusula GROUP BY es que, además de agrupar, **permite ordenar los registros de salida**. Por defecto, si tenemos una cláusula GROUP BY, **las filas de la salida serán ordenadas por las expresiones utilizadas en el GROUP BY**, pudiendo utilizar los modificadores ASC o DESC como si fuera un ORDER BY. Si quisiéramos ordenar las filas por algún otro campo, se puede añadir un ORDER BY de manera explícita. **Si no se deseara ordenar el resultado, habría que añadir la cláusula ORDER BY NULL.**

Sin embargo, en otros SGBD esto no es posible e incluso si lo era, actualmente está considerado como obsoleto, por lo que **se recomienda la ordenación de una salida a través de la cláusula ORDER BY en vez de directamente con GROUP BY.**

Ejemplos:

Para visualizar los números de departamento y el número de empleados que hay en cada uno, ordenándolos de manera descendente, realizaríamos:

```
SELECT dept_no, COUNT(*) FROM emple GROUP BY dept_no DESC;
```

dept_no	COUNT(*)
30	6
20	5
10	3
0	1

Para visualizar el mayor sueldo de cada uno de los oficios, podríamos hacer.

```
SELECT oficio, MAX(salario) FROM emple GROUP BY oficio ORDER BY 2 DESC;
```

oficio	MAX(salario)
PRESIDENTE	4100
DIRECTOR	3005
ANALISTA	3000
EMPLEADO	2000
VENDEDOR	1625

3. Condición de agrupamiento (Having)

A veces, se desea **restringir el resultado de una expresión agrupada**. Sin embargo, se obtendrá error si, por ejemplo, escribimos:

```
SELECT tipo, modelo, cantidad, SUM(Cantidad)
FROM existencias
WHERE SUM(Cantidad)>500
GROUP BY tipo, modelo;
```

El SGBD devolvería este **error**: *WHERE SUM(Cantidad)>500 ERROR en línea 3: función de grupo no permitida aquí*

La razón es que **el SGBD calcula primero el WHERE y luego los grupos**; por lo que esa condición no la puede realizar al no estar establecidos los grupos.

Para expresar condiciones sobre los grupos se utiliza la cláusula **HAVING**, que se ejecuta una vez realizados los grupos. **Indica que, después de haber formado los grupos de filas, se descarten aquellos grupos que no cumplan la condición expresada con un predicado**. Se usaría de esta forma:

```
SELECT tipo, modelo, SUM(Cantidad)
FROM existencias
GROUP BY tipo, modelo
HAVING SUM(Cantidad)>500;
```

Un ejemplo similar, sobre la tabla *Emple* mostraría sólo los grupos de oficios por departamento cuya suma de salarios sea mayor que 2500:

```
SELECT oficio, dept_no, SUM(salario)
FROM emple
GROUP BY oficio, dept_no
HAVING SUM(salario)>2500;
```

oficio	dept_no	SUM(salario)
ANALISTA	20	6000
DIRECTOR	10	2885
DIRECTOR	20	2900
DIRECTOR	30	3005
PRESIDENTE	10	4100
VENDEDOR	30	6075

La cláusula **WHERE** se usa para expresar una o varias condiciones sobre las filas origen, filas sobre las que se harán los grupos. Después, sobre esos grupos se seleccionan los que cumplan las condiciones de la cláusula **HAVING**.

✓ Por ejemplo, esta expresión **sí sería válida**:

```
SELECT tipo, modelo, SUM(Cantidad)
FROM existencias
WHERE tipo != 'AR'
GROUP BY tipo, modelo
HAVING SUM(Cantidad)>500;
```

3. Condición de agrupamiento (Having)

En resumen, los pasos en la ejecución de una instrucción de agrupación por parte del sistema gestor de bases de datos son:

1. Seleccionar las filas deseadas utilizando **WHERE**. Esta cláusula eliminará columnas en base a la condición indicada.
2. Se establecen los grupos indicados en la cláusula **GROUP BY**.
3. Se calculan los valores de las funciones de totales (**COUNT, SUM, AVG,...**).
4. Se filtran los registros que cumplen la cláusula **HAVING**.
5. El resultado se ordena en base al apartado **ORDER BY**.

La cláusula **HAVING** puede utilizarse también sin un **GROUP BY** previo. En este caso, se aplica la condición al único grupo de filas formado por todas las filas de la tabla resultante. En esencia, podemos ver la cláusula **HAVING** al **GROUP BY** como la **WHERE** al **SELECT**, es decir, son filtros que se aplican para obtener los resultados deseados.

RESUMEN DE CONSIDERACIONES IMPORTANTES

- En una consulta de agrupación sólo se puede incluir en las columnas de salida: literales (constantes), las funciones de agrupación (**COUNT, SUM, AVG, MAX, MIN,...**) y las columnas que aparezcan en **GROUP BY**.
 - En MariaDB, en la cláusula **SELECT** pueden figurar expresiones que no utilicen funciones de agregación y columnas que no aparecen en la cláusula **GROUP BY**, aspecto que está totalmente prohibido en otros SGBD.
 - Esta posibilidad de MariaDB puede ser interesante y simplifica la escritura de la consulta, a la vez que mejora el rendimiento, cuando la expresión en cuestión es única para cada grupo. En cambio, se desaconseja este método si la expresión en cuestión no es única en el grupo ya que MariaDB devolverá cualquier valor para el grupo y el resultado obtenido no tiene mucho sentido en principio.
- Sobre un ejemplo:** La cláusula **SELECT emple.dept_no, depart.dnombre, COUNT(*) AS num_emple FROM emple JOIN depart ON (emple.dept_no) GROUP BY emple.dept_no ORDER BY num_emple DESC;**
- Está correctamente escrita y tiene sentido, a pesar de agrupar sólo por el campo **emple.dept_no** y no por el resto de campos que aparecen en la cláusula **SELECT**. Está bien porque para cada grupo de agrupación (**dept_no**) sólo hay un nombre (**dnombre**).

dept_no	dnombre	num_emple
30	VENTAS	6
20	INVESTIGACION	5
10	CONTABILIDAD	3

3. Condición de agrupamiento (Having)

RESUMEN DE CONSIDERACIONES IMPORTANTES

En cambio, si se especificase *SELECT emple.dept_no, emple.oficio, COUNT(*) AS num_emple FROM emple GROUP BY emple.dept_no ORDER BY num_emple DESC*; nos encontraríamos con resultado ambiguo, pues la columna oficio no tiene un valor único para cada fila de departamento y, por ejemplo, los 6 empleados del departamento 30, no son todos vendedores, como muestra la salida.

dept_no	oficio	num_emple
30	VENDEDOR	6
20	EMPLEADO	5
10	DIRECTOR	3
0	EMPLEADO	1

- Para AVG, MAX, MIN y SUM, el resultado tiene el mismo tipo de datos que el argumento. Éste debe ser numérico para AVG y SUM y puede ser de cualquier tipo para MAX y MIN.
- Si se desea contar sin duplicados, se utilizaría COUNT(DISTINCT campo).
- Por defecto, si hay sentencia GROUP BY, las filas de la salida se ordenarán por las expresiones utilizadas en la sentencia GROUP BY. Se puede especificar ASC ó DESC después de esas expresiones.
 - Aunque esta especificación sigue siendo válida en MariaDB, ya se ha comentado que para otros SGBD esto no es posible realizarlo o se está considerando obsoleto, por lo que se recomienda siempre la ordenación utilizando la sentencia ORDER BY oportuna.
- Las funciones de agregación sólo se pueden usar en las cláusulas SELECT y HAVING, es decir, si se intenta hacer uso de una función de agregación en la cláusula WHERE, se obtendrá un error.

***HACER EJERCICIOS DEL 1 AL 14**

4. Operaciones entre consultas

Operador Union	Operador Intersect	Operador Except
El operador UNION permite añadir el resultado de una SELECT a otra SELECT. Para ello, ambas instrucciones tienen que utilizar el mismo número y tipo de columnas. Ejemplo: <pre>SELECT nombre FROM provincias UNION SELECT nombre FROM comunidades</pre> El resultado es una tabla que contendrá nombres de provincia y de comunidades. Es decir, UNION crea una sola salida con registros que estén presentes en cualquiera de las consultas. Si están repetidas sólo aparecen una vez, para mostrar los duplicados se utiliza UNION ALL en lugar de la palabra UNION. Es muy importante señalar que tanto esta cláusula como el resto de combinaciones especiales, requieren que las dos SELECT que unen tengan el mismo tipo de columnas (y en el mismo orden).	De la misma forma, el operador INTERSECT permite unir dos consultas SELECT de modo que el resultado serán las filas que estén presentes en ambas consultas. Ejemplo: tipos y modelos de piezas que se encuentren sólo en los almacenes 1 y 2: <pre>SELECT tipo,modelo FROM existencias WHERE n_almacen=1 INTERSECT SELECT tipo,modelo FROM existencias WHERE n_almacen=2</pre>	El operador EXCEPT selecciona filas de la primera tabla y elimina de ellas las filas que también se encuentran en la segunda tabla. Ejemplo: mostrar los clientes que no son empleados. <pre>SELECT e_nombre AS nombre, email FROM clientes EXCEPT SELECT c_nombre AS nombre, email FROM empleados;</pre> Los operadores INTERSECT y EXCEPT se recogen en las últimas versiones de MariaDB. En versiones anteriores, habría que “simular” estas operaciones utilizando otros operadores.

***HACER EJERCICIOS DEL 15 AL 21**

***PARA REPASAR LAS AGRUPACIONES: HACER EJERCICIOS DEL 22 AL 37**